SINERGIJA 2026 · FACILITATOR GUIDE

From Laptop to Landing Zone

Migrating and modernizing Ledgerly to Azure

2-hour workshop | Presenter-led demo | For system administrators

Discover → Diagram → Migrate → Modernize → Break & fix

Sinergija 2026

Presenter-led workshop · open-source demo repository

github.com/marinfrankovic/Sinergija2026_Workshop

Facilitator guide · preparation, run-of-show, off-screen prompts, recovery and cleanup

OVERVIEW

At a glance

Two hours, five live blocks. Keep this page next to the laptop. Copilot works in the background: the prompts are in this guide, not on the projector.

Time	Block	Outcome (one line)	Checkpoint tag
0–8	**Intro**	Ledgerly running “on-prem” at `http://localhost:8081`, green badge, story set.	`00-onprem`
8–28	**1 Discover**	Inventory + ranked risks from real data (`02-discover.ps1`).	`01-discovered`
28–43	**2 Diagram**	As-is and to-be diagram drawn by Copilot with the draw.io MCP server.	`02-diagrammed`
43–70	**3 Migrate**	What-if clean, data moved 5.7 → 8.0, same UI running on Azure.	`03-migrated`
70–90	**4 Modernize**	API reaches MySQL with its managed identity — no DB password.	`04-modernized`
90–110	**5 Break & fix**	Three faults diagnosed from telemetry, each fixed by script.	`05-broken` / `06-fixed`
110–120	**Wrap-up**	Resource Graph, cost by tags, Monday checklist, Q&A.	—

How to read each block section

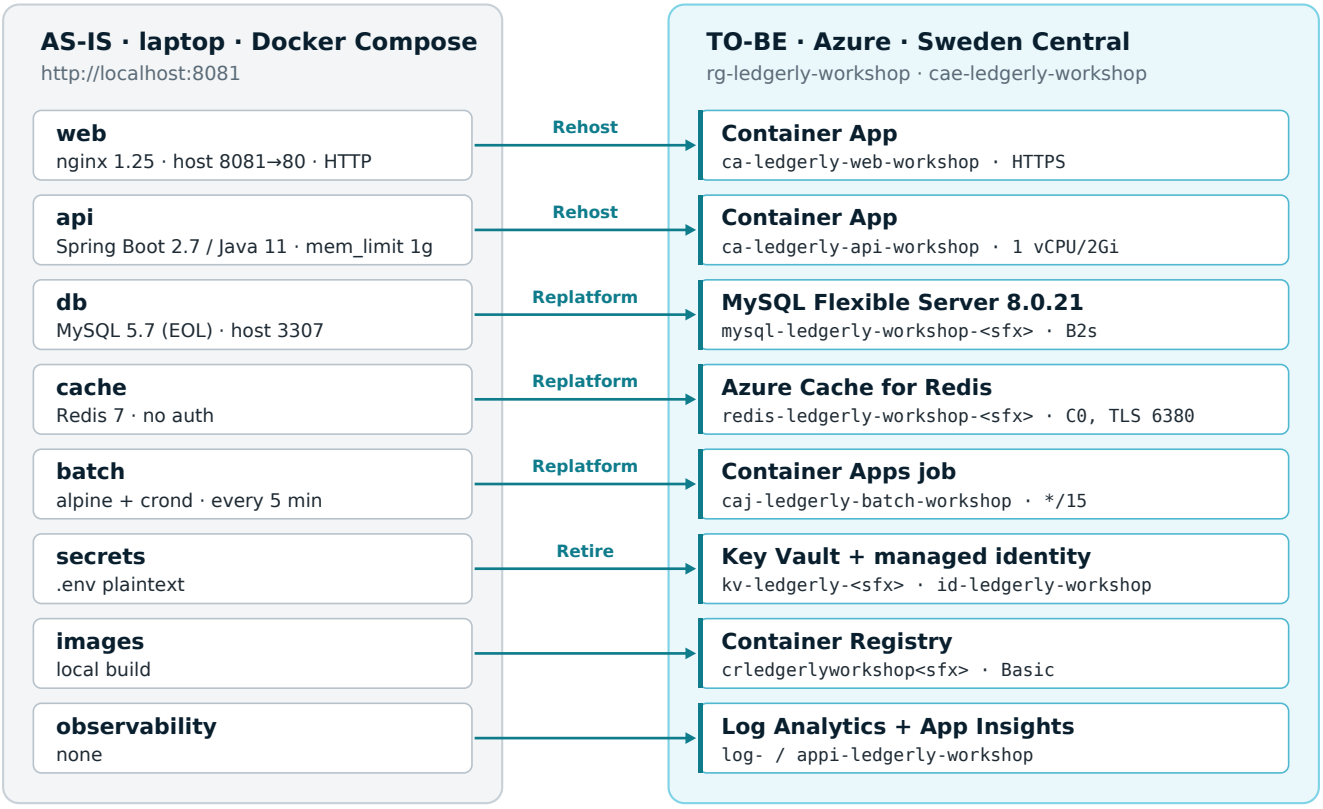
Box	Use it for
Numbered steps	What you do and type, in order. Commands are in grey boxes — copy them as is.
Prompt (off-screen)	Paste into Copilot Chat (Agent mode) on your second screen. Show only the result.
Expected result	What the audience should see. If you see it, move on.
Copilot got it wrong (on purpose)	The likely mistake, and the tool output that catches it. Make this moment visible.
If it goes wrong	Fast recovery. Do not debug on stage — jump to the checkpoint tag.
Checkpoint tag	Git tag that holds the files for the end of the block. `git checkout <tag>`, back with `git checkout main`.

> **i Note**
> Tags capture **files**, not Azure state. Azure state is restored with `azd provision`, `./scripts/reset.ps1` and the fix scripts.
> Data: 12 portfolios, 30 instruments, 360 positions, **200,000 transactions** (2024-01-01 … 2025-12-31). All secrets in the repo are fake.

OVERVIEW

Architecture: as-is vs to-be

Ledgerly is a fictional internal portfolio ledger & reporting app. “On-prem” means Docker Compose on the presenter laptop. The target is Azure Container Apps plus managed data services, deployed with Bicep + azd.



Flows on-prem: browser → web:80 (host 8081) → api:8080 → db:3306 (JDBC, no TLS) · api → cache:6379 · batch → db:3306 → /reports/*.csv. On Azure, web has external HTTPS ingress, api has internal ingress on 8080, MySQL and Redis use TLS (3306/6380).

> **i Note**
> `<sfx>` = 5-character `uniqueString(subscription, env, location)`. Read the real names with `azd env get-values`.

Ports & URLs

What	Where
Ledgerly on-prem web	`http://localhost:8081` (compose `8081:80`, HTTP only)
draw.io MCP server	`http://localhost:8080/mcp` · health `http://localhost:8080/health`
On-prem MySQL	localhost:`3307` → 3306 (legacy smell)
api / cache	internal only (8080 / 6379)
Azure web	`azd env get-value WEB_URI` → `https://ca-ledgerly-web-workshop.<env-domain>.swedencentral.azurecontainerapps.io`
API endpoints	`GET /api/health` (always 200; status UP/DEGRADED) · `GET /api/portfolios` · `GET /api/portfolios/{id}/positions` · `GET /api/reports/monthly?year=2025&month=6` (slow) · `POST /api/reports/run`

OVERVIEW

Resource names

Exact names for azd environment `workshop` in region `swedencentral`. Use them verbatim in prompts and commands.

Thing	Value
Resource group	`rg-ledgerly-workshop` — tags: owner, environment, costCenter, workload=ledgerly
Container Apps env	`cae-ledgerly-workshop` — Consumption, logs → Log Analytics
Web app	`ca-ledgerly-web-workshop` — external HTTPS ingress → 80, URL = `WEB_URI`
API app	`ca-ledgerly-api-workshop` — internal ingress → 8080, 1 vCPU / 2Gi, scale 1–5 @ 50 concurrent
Batch job	`caj-ledgerly-batch-workshop` — schedule `*/15 * * * *`, `RUN_ONCE=true`
MySQL	`mysql-ledgerly-workshop-<sfx>` — 8.0.21, Burstable B2s, 32 GB, db `ledgerly`, admin `ledgerlyadmin`
MySQL firewall	`AllowAllAzureServices` (0.0.0.0) + `AllowPresenterIp`
MySQL params	`slow_query_log=ON`, `long_query_time=2`, `log_queries_not_using_indexes=ON`; diagnostics → `MySqlSlowLogs`
Redis	`redis-ledgerly-workshop-<sfx>` — Basic C0, TLS 6380 (alt. Azure Managed Redis)
Key Vault	`kv-ledgerly-<sfx>` — RBAC, secrets `mysql-admin-password`, `redis-password`
Container Registry	`crledgerlyworkshop<sfx>` — Basic, admin disabled
Identities	`id-ledgerly-workshop` (apps: AcrPull, KV Secrets User, MySQL Entra user) · `id-ledgerly-mysql-workshop` (MySQL server, Entra lookup)
Monitoring	`log-ledgerly-workshop`, `appi-ledgerly-workshop`
Placeholder images	`mcr.microsoft.com/k8se/quickstart:latest`, jobs `mcr.microsoft.com/k8se/quickstart-jobs:latest`

PREPARATION

The day before

MySQL Flexible Server takes minutes to create, and one prerequisite needs a tenant admin. Do everything on this page the day before — nothing here happens on stage.

Laptop prerequisites

- [ ] Windows laptop with **Docker Desktop** (Linux containers, ≥ 4 GB RAM), **PowerShell 7**, **VS Code** (recommended extensions in `.vscode/extensions.json`), **Azure CLI** (+ Bicep), **Azure Developer CLI**, **git**, **Node.js** (Azure MCP Server via `npx`), GitHub Copilot (Agent mode).
- [ ] Azure subscription with Owner (or Contributor + User Access Administrator).
- [ ] Run `./scripts/00-prereqs.ps1` (add `-Register` to register resource providers).
- [ ] Re-create the draw.io MCP container bound to loopback only — the endpoint has no authentication, and the old container maps `8080:8080` on all interfaces:

```
docker rm -f drawio-mcp
docker run -d --name drawio-mcp -p 127.0.0.1:8080:8080 simonkurtzmsft/drawio-mcp-server:latest
```

Tenant / Entra prerequisite (needs a tenant admin)

- [ ] Grant the MySQL server identity `id-ledgerly-mysql-workshop` *User.Read.All, GroupMember.Read.All, Application.Read.All* (or the *Directory Readers* role). Without it block 4 fails at `CREATE AADUSER`.
- [ ] Set `MYSQL_ENTRA_ADMIN_LOGIN` — if it is empty, no Entra admin is created.

Pre-provision Azure

1. **Sign in and create the environment**

```
az login ; azd auth login
azd env new workshop                                    # -> rg-ledgerly-workshop
azd env set AZURE_LOCATION swedencentral
azd env set MYSQL_ADMIN_PASSWORD '<your password>'
azd env set PRESENTER_IP (Invoke-RestMethod https://api.ipify.org)
azd env set MYSQL_ENTRA_ADMIN_LOGIN (az ad signed-in-user show --query userPrincipalName -o
    tsv)
```

2. **Check the templates compile, then what-if**

```
az bicep build -f infra/main.bicep
docker compose build
azd provision --preview        # what-if — ALWAYS first
```

3. **azd up, then azd provision again**

```
azd up           # provision (placeholder images) + deploy web/api + postdeploy hook builds
    batch
azd provision    # 2nd run: real image names now known -> api probes enabled, no placeholder
    drift
```

The first provision runs public placeholder images because ACR is still empty. If the first provision fails while role assignments propagate, simply run `azd provision` again.

Rehearse and create checkpoints

4 Rehearse the Azure blocks end to end

```
./scripts/03-migrate-data.ps1
./scripts/04-modernize.ps1
./scripts/faults/fault-1-oom.ps1 ; ./scripts/faults/fix-1-oom.ps1
./scripts/faults/fault-2-network.ps1 ; ./scripts/faults/fix-2-network.ps1
./scripts/faults/fault-3-slowquery.ps1 ; ./scripts/faults/fix-3-slowquery.ps1
```

Check the fault-1 restart loop really happens. Fault-3 timings depend on B2s burst credits.

5 Reset Azure to the pre-show state

```
azd provision          # undo block 4 (password-based api config again)
./scripts/reset.ps1    # re-arm faults: firewall rule, api 1 vCPU/2Gi, JAVA_OPTS, index
```

Optionally keep the data migrated; the MySQL Entra user stays (harmless). Local fault-3 rehearsal: add `-Local`.

6 Create checkpoint tags and the icon library

```
./scripts/make-checkpoints.ps1          # once, on a fresh presentation copy
./scripts/build-drawio-library.ps1      # -> docs/diagrams/ledgerly-icons.xml
```

PREPARATION

30 minutes before

Work top to bottom. Every box ticked means a calm start.

Containers

- ☐ **Docker Desktop** running (Linux containers).
- ☐ **drawio-mcp** container running and healthy; Ledgerly on-prem up (`docker compose up -d --build`, waits for health, opens the browser):

```
docker start drawio-mcp         # created the day before (bound to 127.0.0.1:8080)
curl http://localhost:8080/health
./scripts/01-onprem-up.ps1      # -> http://localhost:8081, green badge UP · db UP · cache UP
```

Azure sign-in

- ☐ `az login`, `azd auth login`, `azd env select workshop`.
- ☐ `./scripts/00-prereqs.ps1` all OK.
- ☐ Public IP unchanged? If not: `azd env set PRESENTER_IP (Invoke-RestMethod https://api.ipify.org)` then `azd provision`.

VS Code and screen

- ☐ MCP servers **drawio**, **azure**, **thesvg** started (from `.vscode/mcp.json`); Copilot Chat in **Agent mode**.
- ☐ Font size up in editor and terminal; notifications off; this guide on the second screen for the off-screen prompts.
- ☐ Portal tabs: `rg-ledgerly-workshop`, Log Analytics (queries from `kql/queries.kql` pasted), App Insights failures, Container App revisions.

> **i Note**
> **Backup plan:** `git tag` lists `00-onprem` … `06-fixed` · pre-built diagrams in `docs/diagrams/` open in `hediet.vscode-drawio` · Bicep visualizer on `infra/main.bicep` · `docs/inventory.md` is the expected discovery answer.

BLOCK 0

Intro 0–8 min

Goal: set the story. Ledgerly is a legacy app on a laptop; by the end it runs on Azure, passwordless and observable.

1. **Show the app running “on-prem”** 2 min

   Open `http://localhost:8081` in the browser. Point at the 12 portfolios and the green health badge.

   > **✓ Expected result**
   > UI with 12 portfolios, green badge `UP · db UP · cache UP`.

2. **Tell the story (slide)** 4 min

   A fictional internal portfolio ledger & reporting app: Java/Spring Boot + MySQL 5.7 + Redis + nginx + a cron batch, all in Docker Compose. Today: **discover** → **diagram** → **migrate** → **modernize** → **break & fix**.

3. **Set expectations about Copilot** 2 min

   Copilot works in the background; you show results, not typing. It will make mistakes today — and the tools will catch them.

   > **✖ Copilot got it wrong (on purpose)**
   > No planned mistake in this block. Announce the pattern: every block has one moment where Copilot is wrong and a script, linter or query proves it.

> **⚠ If it goes wrong**
> Badge not green: the first start seeds 200,000 transactions (1–2 min). Then check `docker compose ps` and `docker compose logs api db`.
> Still broken: `git checkout 00-onprem` and re-run `./scripts/01-onprem-up.ps1`.

> **⚑ Checkpoint tag**
> `00-onprem` — app, compose, `.env`, prereqs + up scripts, Copilot/MCP config. No Azure state needed.

BLOCK 1

Discover 8–28 min

Goal: build an inventory and a ranked risk list from real data — then let Copilot do the same and compare.

1 Run discovery 6 min

```
./scripts/02-discover.ps1      # writes docs/discovery-output.txt
```

Walk through the sections: compose services, images + versions, env var names (secrets masked), port map, volumes, docker stats, MySQL version, row counts, indexes on `transactions`, JAVA_OPTS vs memory limit.

> **✓ Expected result**
> 5 services; `mysql 5.7.x`; 200,000 rows; secrets masked.
> **RISK: -Xmx (2048 MiB) >= container limit (1024 MiB)**.
> `transactions` only has the `PRIMARY` index.

2 Ask Copilot for the inventory 5 min

PROMPT (OFF-SCREEN) · paste into Copilot Chat, Agent mode

```
Read docker-compose.yml, .env and both Dockerfiles. Produce an inventory table: service,
    image, version, ports, volumes, env vars (mask secrets), and the top 5 migration risks.
```

Show Copilot's table next to the script output.

3 Compare with the reference inventory 5 min

Open `docs/inventory.md`. Ranked risks to land:

#	Risk
1	**MySQL 5.7 end-of-life** — no security fixes; move to 8.0.
2	**Plaintext secrets** in `.env`; DB published on host port 3307.
3	**JVM heap > container limit** (`-Xmx2g` vs `mem_limit 1g`) → OOMKilled under load.
4	**Slow monthly report** — N+1 queries, full scans (no `booking_date` index).
5	**Silent batch failures** (`> /dev/null`, `|| true`).
6	**Legacy runtime** — Spring Boot 2.7 on Java 11.

4 Dig into two risks 4 min

PROMPT (OFF-SCREEN) · paste into Copilot Chat, Agent mode

```
Compare the api JAVA_OPTS with the container memory limit. What happens under load?
```

PROMPT (OFF-SCREEN) · paste into Copilot Chat, Agent mode

```
Which MySQL 5.7 → 8.0 incompatibilities should I check in db/init/*.sql?
```

Plant the seed: the JAVA_OPTS risk comes back as fault 1, the missing index as fault 3.

> **✖ Copilot got it wrong (on purpose)**
> Copilot reports the web port as **8080** (container port) instead of host **8081**, or misses that MySQL 5.7 is end-of-life.
> **Caught by:** the port map and version output of `02-discover.ps1`.

> **⚠ If it goes wrong**
> Discovery script or Copilot stalls: `git checkout 01-discovered` and present `docs/inventory.md` as the answer.

> **⚑ Checkpoint tag**
> `01-discovered` — adds `scripts/02-discover.ps1` and `docs/inventory.md`.

BLOCK 2

Diagram 28–43 min

Goal: Copilot draws the as-is and to-be architecture through the draw.io MCP server (700+ embedded Azure icons), with brand icons from thesvg. The server is **stateless**: every call must carry the previous `diagram_xml` forward. Setup: Copilot Chat in **Agent mode**, tools `drawio` + `thesvg` enabled, `#file:docker-compose.yml` in context.

1 Shapes, groups and cells for the to-be page 5 min

PROMPT (OFF-SCREEN) · paste into Copilot Chat, Agent mode

```
Use the drawio tools: search-shapes for all Azure icons in one call, then create-groups for
    rg-ledgerly-workshop and cae-ledgerly-workshop, then add-cells in one batch using temp_id
    edges. Use exact names from infra/main.bicep.
```

The `search-shapes` call should carry all queries in one array:

```
queries: ["container apps environment", "container apps", "mysql flexible",
          "cache redis", "key vaults", "container registries", "managed identities",
          "log analytics", "application insights", "resource groups"]
```

2 As-is page with brand icons 5 min

PROMPT (OFF-SCREEN) · paste into Copilot Chat, Agent mode

```
For the as-is page use thesvg icons for docker, nginx, mysql, redis and openjdk as embedded
    image cells.
```

Laptop / Docker host boundary via `create-groups`. Icons: `C:\Github\tools\thesvg\public\icons\<slug>\default.svg`, the `thesvg` MCP server, or `docs/diagrams/ledgerly-icons.xml`. Style (no `;base64` — the `;` breaks the style string):

```
shape=image;image=data:image/svg+xml,<base64>;aspect=fixed
```

3 Check, export, save, tidy 5 min

PROMPT (OFF-SCREEN) · paste into Copilot Chat, Agent mode

```
get-diagram-stats, export-diagram, save to docs/diagrams/ledgerly-live.drawio.
```

Open `docs/diagrams/ledgerly-live.drawio` in the draw.io VS Code editor (`hediet.vscode-drawio`), tidy by hand, commit.

✔ Expected result

Two pages: as-is and to-be. `get-diagram-stats` shows the expected pages, cell and edge counts.

✖ Copilot got it wrong (on purpose)

Copilot forgets to pass `diagram_xml`, so the second call silently starts from an empty diagram; or it invents icon names.

Caught by: the `get-diagram-stats` cell count, the `search-shapes` results, and `dry_run` on `add-cells`.

⚠ If it goes wrong

No MCP / no network: `git checkout 02-diagrammed`, open the pre-built `ledgerly-as-is.drawio` / `ledgerly-to-be.drawio`. Second fallback: *Bicep: Open Visualizer* on `infra/main.bicep`.

Server down: `curl http://localhost:8080/health`, restart `drawio-mcp`.

⚑ Checkpoint tag

`02-diagrammed` — adds `docs/diagrams/*.drawio`, `docs/drawio-mcp.md`, `scripts/build-drawio-library.ps1`.

BLOCK 3

Migrate 43–70 min

Goal: show the landing zone as code, prove the what-if is clean, move the data from MySQL 5.7 to Flexible Server 8.0 and open the same app on Azure. Infra was pre-provisioned the day before.

1 Explain the Bicep 7 min

PROMPT (OFF-SCREEN) · paste into Copilot Chat, Agent mode

```
Explain infra/main.bicep module by module and list every role assignment.
```

6R: web/api **rehost**, db/cache/batch **replatform**, `.env` secrets **retire** (Key Vault + managed identity).

2 What-if 5 min

```
azd provision --preview
```

PROMPT (OFF-SCREEN) · paste into Copilot Chat, Agent mode

```
Run azd provision --preview and summarise the what-if.
```

✓ Expected result

What-if: **no changes** (pre-provisioned).

Optional, if time allows: `azd deploy web` live (remote build in ACR).

3 Migrate the data 10 min

PROMPT (OFF-SCREEN) · paste into Copilot Chat, Agent mode

```
Write the mysqldump command for a 5.7 source going to Azure MySQL 8.0 — which flags matter?
```

```
./scripts/03-migrate-data.ps1
```

5.7 → 8.0 pre-checks, `mysqldump --single-transaction --routines --triggers --set-gtid-purged=OFF`, import via a temporary `mysql:8.0` client (`--ssl-mode=REQUIRED`), row counts on both sides.

✓ Expected result

Row-count table all `True`; target version `8.0.21`.

4 Open Ledgerly on Azure 5 min

```
azd env get-value WEB_URI      # open it: same data (12 portfolios), now over HTTPS
```

Say it: a real cutover adds `-FreezeSource` (stops on-prem api + batch); rollback = `docker compose start api batch`.

✖ Copilot got it wrong (on purpose)

Copilot suggests a preview/old API version or a non-existent property (e.g. MySQL `authConfig` copied from PostgreSQL).

Caught by: Bicep extension squiggles, `az bicep build -f infra/main.bicep`, `azd provision --preview`.

⚠ If it goes wrong

IP changed → `azd env set PRESENTER_IP …` + `azd provision`. Password missing → `azd env set MYSQL_ADMIN_PASSWORD …`

Out of time → `git checkout 03-migrated`, open `WEB_URI` (data kept from rehearsal; the script is repeatable).

⚑ Checkpoint tag

`03-migrated` — adds `azure.yaml`, `infra/`, hook, migrate script, plan. Azure: provisioned, deployed, data migrated.

BLOCK 4

Modernize 70–90 min

Goal: the API connects to MySQL with its user-assigned managed identity `id-ledgerly-workshop` (Microsoft Entra ID). No DB password in the app any more.

1 Ask how passwordless works 3 min

PROMPT (OFF-SCREEN) · paste into Copilot Chat, Agent mode

```
How do I make the Spring Boot 2.7 api connect to Azure MySQL without a password using the
    user-assigned identity?
```

2 Run the modernize script 7 min

```
./scripts/04-modernize.ps1
```

What it does: gets an Entra token for MySQL as the presenter (Entra admin), creates the MySQL user for the identity, then switches the api app to profile `azure` (simplified):

```
SET aad_auth_validate_oids_in_tenant = OFF;
CREATE AADUSER 'id-ledgerly-workshop' IDENTIFIED BY '<client id>';
GRANT SELECT, INSERT, UPDATE, DELETE ON `ledgerly`.* TO 'id-ledgerly-workshop'@'%';

az containerapp update --set-env-vars SPRING_PROFILES_ACTIVE=azure AZURE_CLIENT_ID=<client id>
    DB_USER=id-ledgerly-workshop
    --remove-env-vars DB_PASSWORD
az containerapp secret remove --secret-names db-password
```

✓ Expected result
`CREATE AADUSER` ok; new revision (~60 s to start); health `UP`; env list without `DB_PASSWORD`.

3 Prove it 5 min

```
az containerapp show -n ca-ledgerly-api-workshop -g rg-ledgerly-workshop --query
    "properties.template.containers[0].env[].name" -o tsv
```

PROMPT (OFF-SCREEN) · paste into Copilot Chat, Agent mode

```
Which secrets can now be removed from the api Container App?
```

Show Key Vault `kv-ledgerly-<sfx>` (RBAC; secrets `mysql-admin-password`, `redis-password`). Note: the batch job still uses the admin password from Key Vault — next modernization step.

4 App modernization talk 5 min

PROMPT (OFF-SCREEN) · paste into Copilot Chat, Agent mode

```
What would an upgrade to Spring Boot 3 / Java 21 involve for this pom.xml?
```

Talk track: Java 17/21, Spring Boot 3 — the Replatform/Refactor path from the migration plan.

✖ Copilot got it wrong (on purpose)
Copilot uses `CREATE USER … IDENTIFIED WITH aad_auth`, or the wrong client id / object id.
Caught by: `04-modernize.ps1` output, `SELECT user, plugin FROM mysql.user`, and the health badge.

⚠ If it goes wrong
`CREATE AADUSER` fails: Graph permissions for `id-ledgerly-mysql-workshop` are missing (day-before prerequisite). If the user already exists the script continues; re-run with `-SkipDbUser` to only switch the app.
Health stays DEGRADED: undo with `azd provision` (re-applies the password-based config) and continue with block 5.
Files only: `git checkout 04-modernized`.

⚑ Checkpoint tag
`04-modernized` — adds `scripts/04-modernize.ps1`. Azure: api on profile `azure`, no `DB_PASSWORD`.

BLOCK 5

Break & fix 90–110 min

Goal: three realistic faults, one at a time (about 6 minutes each). Pattern every time: **symptom → evidence → root cause → fix script**. Let the telemetry, not a guess, name the cause.

> **i Note**
> Keep the Log Analytics tab ready with the queries from `kql/queries.kql` (Appendix A). The Container Apps environment writes to `*_CL` tables; if a query returns nothing, run the non-`_CL` variant.

Fault 1 — OOM ("right-sized to save money") ~6 min

⚡ Inject

```
./scripts/faults/fault-1-oom.ps1      # api 0.5 vCPU/1Gi, JAVA_OPTS=-Xms1g -Xmx2g
    -XX:+AlwaysPreTouch
```

Symptom 502 / badge "API DOWN"; the revision never becomes healthy.

Evidence

```
az containerapp replica list -n ca-ledgerly-api-workshop -g rg-ledgerly-workshop
    --query "[].{name:name, restarts:properties.containers[0].restartCount,
    state:properties.runningState}" -o table
az containerapp logs show -n ca-ledgerly-api-workshop -g rg-ledgerly-workshop --type
    system --tail 30
```

restartCount climbs. Log Analytics: **KQL #1** (OOMKilled / exit 137).

> **PROMPT (OFF-SCREEN) · paste into Copilot Chat, Agent mode**
> `Why is my container app ca-ledgerly-api-workshop restarting?`

Root cause JVM heap hard-coded above the container memory (`-Xmx2g` on 1Gi) — the risk found in Discover.

Fix

```
./scripts/faults/fix-1-oom.ps1        # 1 vCPU/2Gi +
    JAVA_OPTS=-XX:MaxRAMPercentage=75.0
```

> **✔ Expected result**
> Restart count of the new revision stays at 0. Permanent fix: remove `ENV JAVA_OPTS="-Xms1g -Xmx2g"` from `api/Dockerfile`.

> **✖ Copilot got it wrong (on purpose)**
> Copilot blames MySQL CPU for fault 1, or proposes "just add memory".
> **Caught by:** KQL restart/OOM evidence, the replica restartCount, and `JAVA_OPTS` in the revision env.

Fault 2 — Network ("security hardening gone wrong") ~6 min

⚡ Inject

```
./scripts/faults/fault-2-network.ps1  # deletes AllowAllAzureServices, restarts the
    active api revision
```

Symptom Badge `DEGRADED · db DOWN`; `/api/portfolios` returns 500 after ~5 s.

Evidence

```
az mysql flexible-server firewall-rule list -g rg-ledgerly-workshop -n
    mysql-ledgerly-workshop-<sfx> -o table
```

KQL #2: "Connection is not available, request timed out" / "Communications link failure".
KQL #3: App Insights failed dependencies.

> **PROMPT (OFF-SCREEN) · paste into Copilot Chat, Agent mode**
> `Show failed dependencies for appi-ledgerly-workshop in the last 30 minutes.`

Root cause "Hardening" removed the only path from Container Apps (no VNet) to MySQL's public endpoint.

Fix

```
./scripts/faults/fix-2-network.ps1    # recreates AllowAllAzureServices (0.0.0.0)
```

✔ Expected result

Health badge returns to UP within ~10–20 s. Long term: VNet-integrated environment + MySQL private access, then remove public access.

Fault 3 — Slow query (month-end load) ~6 min

⚡ Inject

```
./scripts/faults/fault-3-slowquery.ps1   # 40 calls to /api/reports/monthly, 8 in
    parallel
```

Symptom

The report button spins; seconds per call.

Evidence

KQL #4: requests > 2000 ms. **KQL #5**: `AzureDiagnostics` `MySqlSlowLogs` (rows_examined ≈ 200k). EXPLAIN shows `type=ALL`:

```
EXPLAIN SELECT COUNT(*) FROM transactions WHERE portfolio_id = 1
  AND booking_date >= '2025-06-01' AND booking_date < '2025-07-01'
```

PROMPT (OFF-SCREEN) · paste into Copilot Chat, Agent mode

```
Which queries in the MySQL slow log examine the most rows?
```

Root cause

No index on `transactions.booking_date` + N+1 queries in the report.

Fix

```
./scripts/faults/fix-3-slowquery.ps1
# CREATE INDEX ix_transactions_booking_date ON transactions (booking_date)
```

✔ Expected result

The script shows EXPLAIN before/after: `type=range`, key `ix_transactions_booking_date`; one report call re-timed.

⚠ If it goes wrong

A fault does not show: wait for the new revision, then check again; fault-3 timings depend on B2s burst credits (the slow log still has evidence).

Stuck mid-fault: run the matching `fix-N` script and move on; files for all fixes are at `git checkout 06-fixed`.

Re-arm everything for the next run: `./scripts/reset.ps1`.

⚑ Checkpoint tag

`05-broken` — fault scripts + `kql/queries.kql`. `06-fixed` — fix scripts, the index migration, reset/cleanup.

WRAP-UP

Wrap-up 110–120 min

Goal: show what was built, what it costs, and what attendees can do on Monday.

1 Everything in one query (Resource Graph) 3 min

```
az graph query -q "resources | where resourceGroup =~ 'rg-ledgerly-workshop' | project name,
    type, location, sku, tags"
```

2 Cost by tags 2 min

Cost Management → *group by tag* `costCenter` / `workload`. Pricing Calculator: B2s MySQL, C0 Redis, ACR Basic, Consumption Container Apps.

3 Talking points 3 min

Block	Line to land
Discover	Inventory from data first; Copilot second — and compare.
Diagram	Diagrams drawn by tools, with exact resource names and real icons.
Migrate	Infra as code, `--preview` always first; dump/restore with verified row counts.
Modernize	Managed identity: the API holds no database password.
Break & fix	Symptom → evidence → root cause → fix. Telemetry beats guessing — including Copilot's guesses.

4 Q&A 2 min

Open questions. Leave the Azure web app and the diagrams on screen.

Monday checklist (for attendees)

- ☐ Inventory one legacy app: services, versions, ports, secrets, risks.
- ☐ Find end-of-life runtimes (e.g. MySQL 5.7, Spring Boot 2.7 / Java 11).
- ☐ Check JVM heap settings against container memory limits.
- ☐ Move secrets out of files into Key Vault; prefer managed identity over passwords.
- ☐ Run what-if (`azd provision --preview`) before every deployment.
- ☐ Turn on logs before you need them: Log Analytics, App Insights, MySQL slow log.
- ☐ Tag resources (`costCenter`, `workload`) so cost can be grouped.

After the session: cleanup

```
./scripts/cleanup.ps1               # azd down --purge --force  +  docker compose down -v
./scripts/cleanup.ps1 -IncludeMcp   # also removes the drawio-mcp container
./scripts/cleanup.ps1 -KeepAzure    # only clean up locally
./scripts/cleanup.ps1 -KeepLocal    # only clean up Azure
```

> **€ Cost**
> Tear down right after the session. `--purge` also purges the soft-deleted Key Vault.

REFERENCE

Troubleshooting

Common failures and the fastest fix. Items marked **KG** are known gaps from the README — verify them in rehearsal.

Symptom	Likely cause	Fix
On-prem badge never turns green	First start seeds 200k rows (1–2 min)	Wait; `docker compose ps`, `docker compose logs api db`
Ledgerly not on 8080	8080 is reserved for the draw.io MCP server	Use `http://localhost:8081`
drawio tools missing in Copilot	Container down or MCP server not started	`curl http://localhost:8080/health`; start `drawio` in `.vscode/mcp.json`
Second diagram call is empty	`diagram_xml` not carried forward	Re-run with the previous `diagram_xml`; check `get-diagram-stats`
Icon renders blank (KG 11)	Built-in `img/lib/azure2/...` name differs	Replace the icon in the editor
thesvg slug missing (KG 12)	Slug name differs	`build-drawio-library.ps1 -Search '*spring*'`
First provision fails (KG 7)	Role assignments still propagating	Run `azd provision` again
Bicep errors (KG 1)	Template was never compiled	`az bicep build -f infra/main.bicep` before rehearsal
api runs placeholder / no probes (KG 5)	Image names not fed back yet	Second `azd provision` after `azd up`
Web on Azure cannot reach api (KG 6)	In-environment name resolution	Use the api internal FQDN with `https://` in `API_UPSTREAM`
Migration cannot connect	Presenter IP changed	`azd env set PRESENTER_IP …` + `azd provision`
`CREATE AADUSER` fails (KG 2, 3)	Missing Graph permissions / syntax changed	Grant permissions to `id-ledgerly-mysql-workshop`; re-check Microsoft Learn syntax
api fails on profile `azure` (KG 4)	Spring Cloud Azure 4.19.0 / Boot 2.7.18 unverified	`azd provision` to go back to password config
Batch job errors (KG 8)	MariaDB client vs `caching_sha2_password`	Check job logs (job keeps exiting 0)
No OOM restart loop (KG 9)	Memory headroom differs	Check in rehearsal; `-XX:+AlwaysPreTouch` is set by the fault
Fault 3 not slow enough (KG 9)	B2s burst credits	Use the slow-log evidence (`log_queries_not_using_indexes=ON`)
KQL returns nothing (KG 10)	Column/table names differ	Run the other variant (`_CL` vs non-`_CL`)
`mysql:5.7` will not start (KG 13)	Image is amd64-only	Use an x64 laptop (emulated on ARM)

APPENDIX A

KQL queries

From `kql/queries.kql`. Workspace `log-ledgerly-workshop`. Run the variant that returns data.

#1 Container restarts / OOMKilled (fault 1)

```
ContainerAppSystemLogs_CL
| where TimeGenerated > ago(30m)
| where ContainerAppName_s == "ca-ledgerly-api-workshop"
| where Log_s has_any ("OOMKilled", "exit code 137", "Restart", "BackOff", "Probe", "terminated")
    or Reason_s in ("ContainerTerminated", "ContainerCrashing", "BackOff", "ProbeFailed")
| project TimeGenerated, RevisionName_s, ReplicaName_s, Reason_s, Log_s
| order by TimeGenerated desc
```

#1b Same, non-_CL variant

```
ContainerAppSystemLogs
| where TimeGenerated > ago(30m)
| where ContainerAppName == "ca-ledgerly-api-workshop"
| where Log has_any ("OOMKilled", "exit code 137", "Restart", "BackOff", "Probe", "terminated")
    or Reason in ("ContainerTerminated", "ContainerCrashing", "BackOff", "ProbeFailed")
| project TimeGenerated, RevisionName, ReplicaName, Reason, Log
| order by TimeGenerated desc
```

#1c Restart count per revision (5-min bins)

```
ContainerAppSystemLogs_CL
| where TimeGenerated > ago(1h) and ContainerAppName_s == "ca-ledgerly-api-workshop"
| where Reason_s has "Terminated" or Log_s has "OOMKilled"
| summarize restarts = count() by bin(TimeGenerated, 5m), RevisionName_s
| render timechart
```

#2 Console errors — DB connectivity (fault 2)

```
ContainerAppConsoleLogs_CL
| where TimeGenerated > ago(30m)
| where ContainerAppName_s == "ca-ledgerly-api-workshop"
| where Log_s has_any ("ERROR", "Exception", "Communications link failure", "Connection is not
   available", "timed out")
| project TimeGenerated, RevisionName_s, Log_s
| order by TimeGenerated desc
```

#2b Same, non-_CL variant

```
ContainerAppConsoleLogs
| where TimeGenerated > ago(30m)
| where ContainerAppName == "ca-ledgerly-api-workshop"
| where Log has_any ("ERROR", "Exception", "Communications link failure", "Connection is not
   available", "timed out")
| project TimeGenerated, RevisionName, Log
| order by TimeGenerated desc
```

#3 App Insights failed dependencies (MySQL / Redis)

```
AppDependencies
| where TimeGenerated > ago(30m)
| where Success == false
| summarize failures = count(), sample = any(Data) by DependencyType, Target, ResultCode
| order by failures desc

// classic schema (appi-ledgerly-workshop > Logs):
// dependencies | where timestamp > ago(30m) and success == false
// | summarize failures = count() by type, target, resultCode
```

#4 Slow requests > 2000 ms (fault 3)

```
AppRequests
| where TimeGenerated > ago(30m)
| where DurationMs > 2000
| summarize count(), p50 = percentile(DurationMs, 50), p95 = percentile(DurationMs, 95), max =
    max(DurationMs) by Name
| order by p95 desc

// classic schema:
// requests | where timestamp > ago(30m) and duration > 2000
// | summarize count(), p95 = percentile(duration, 95) by name
```

#5 MySQL slow query log (fault 3)

```
AzureDiagnostics
| where TimeGenerated > ago(30m)
| where ResourceProvider == "MICROSOFT.DBFORMYSQL" and Category == "MySqlSlowLogs"
| project TimeGenerated, Resource, query_time_d, rows_examined_d, rows_sent_d, sql_text_s
| order by query_time_d desc
| take 50
```

#5b Slow-log summary: statements scanning the most rows

```
AzureDiagnostics
| where TimeGenerated > ago(1h) and Category == "MySqlSlowLogs"
| summarize calls = count(), avg_s = avg(query_time_d), rows_examined = sum(rows_examined_d)
    by statement = substring(sql_text_s, 0, 120)
| order by rows_examined desc
```

Resource Graph (Resource Graph Explorer or az graph query -q "...")

```
resources
| where resourceGroup =~ 'rg-ledgerly-workshop'
| project name, type, location, sku, tags
| order by type asc
```

APPENDIX B

Repository map

```
ledgerly/
├─ docker-compose.yml, .env           on-prem stack (legacy)
├─ web/ api/ db/ batch/ cache/        application tiers
├─ azure.yaml, infra/                 azd + Bicep (subscription scope)
├─ scripts/                           PowerShell 7 workshop scripts
│  ├─ 00-prereqs  01-onprem-up  02-discover  03-migrate-data  04-modernize
│  ├─ build-drawio-library  make-checkpoints  reset  cleanup  _common
│  ├─ faults/   fault-{1,2,3}-*.ps1  fix-{1,2,3}-*.ps1
│  └─ hooks/    postdeploy-batch.ps1
├─ kql/queries.kql                    Log Analytics + Resource Graph queries
├─ db/init/  db/migrations/           schema, seed, 001-ix-transactions-booking-date.sql
├─ docs/                              inventory, migration plan, draw.io MCP guide, diagrams/
├─ .vscode/ .github/                  MCP servers (drawio, azure, thesvg), Copilot instructions
└─ checkpoints.md, PRESENTER-FACTS.md
```

All scripts use paths relative to the repo root, so the folder can live anywhere.

APPENDIX C

Icon and diagram tips

draw.io MCP tool flow

```
search-shapes (all Azure queries in ONE queries array)
  -> create-groups (rg-ledgerly-workshop, cae-ledgerly-workshop; laptop boundary on as-is)
  -> add-cells (ONE batch, shape_name icons, edges by temp_id; dry_run to validate)
  -> brand icons from thesvg as image cells
  -> create-page "to-be" -> get-diagram-stats -> export-diagram
  -> save .drawio -> open in VS Code (hediet.vscode-drawio) -> tidy -> commit
```

- [ ] **Stateless:** every call takes the full previous `diagram_xml` and returns a new one.
- [ ] Edge labels for the to-be page: HTTPS, internal HTTP, TLS 3306/6380, secret refs, AcrPull, logs.
- [ ] Keep diagrams self-contained: embed icons as data URIs, never link the CDN or a `file:///` path.

thesvg brand icons

```
C:\Github\tools\thesvg\public\icons\<slug>\<variant>.svg   # variants: default, mono, light, dark,
    wordmark
C:\Github\tools\thesvg\src\data\icons.json                 # manifest

./scripts/build-drawio-library.ps1                         # -> docs/diagrams/ledgerly-icons.xml
./scripts/build-drawio-library.ps1 -Search '*java*','*spring*' -Slugs docker,nginx
```

Default slugs: docker, nginx, mysql, redis, openjdk, spring, github, visualstudiocode, powershell, microsoftazure (missing slugs are skipped with a warning). Open the library via *File › Open Library from › Device*, or drag SVGs from the folder onto the VS Code draw.io canvas. Optional MCP server: `npx -y @thesvg/mcp-server`.